

A PROJECT THESIS ON
IoT-BASED ACCIDENT DETECTION, ALERT AND
AMBULANCE NAVIGATION SYSTEM

A thesis submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING / TECHNOLOGY

in
[Department / Branch Name]

Submitted by
[Student Name(s) and Roll/Registration Number(s)]

Under the guidance of
[Guide / Supervisor Name, Designation]

[Department / College / University Name]
[City] — [Year]

CERTIFICATE

This is to certify that the thesis titled “IoT-Based Accident Detection, Alert and Ambulance Navigation System,” submitted by [Student Name(s)], is a bona fide record of the project work carried out under my/our supervision, in partial fulfilment of the requirements for the award of the degree of [Degree Name] in [Department Name], during the academic year [Year].

[Guide Name and Designation]

[Head of Department]

[Date]

[Date]

DECLARATION

I/We hereby declare that the work presented in this thesis, titled “IoT-Based Accident Detection, Alert and Ambulance Navigation System,” is an authentic record of the work carried out by me/us under the guidance of [Guide Name], and has not been submitted elsewhere for the award of any other degree or diploma.

[Student Name(s)]

[Roll/Registration Number(s)]

[Date]

ACKNOWLEDGEMENT

[Insert acknowledgement text here — typically thanking your project guide/supervisor, department head, institution, and family/peers who supported the project.]

ABSTRACT

Delayed emergency response is one of the largest controllable factors in road-accident mortality. This thesis presents the design and implementation of a low-cost, IoT-based Accident Detection, Alert and Ambulance Navigation System that automatically detects a vehicular accident, alerts a configured emergency contact through redundant channels, preserves video evidence of the event, tracks ambulance positions in real time, and automatically assigns and routes the nearest available ambulance to the accident — using exclusively free, open-source mapping and routing infrastructure (OpenStreetMap and OSRM), with no proprietary mapping API anywhere in the system.

The system comprises three independently networked embedded firmware projects — a Vehicle Unit (ESP32-S3 with an MPU6050 accelerometer/gyroscope, GPS, SIM800L GSM module, and SH1106 OLED display) performing threshold-based crash detection; a Camera Module (Seeed XIAO ESP32-S3 Sense) that continuously buffers video and, on trigger, saves and uploads a clip spanning the accident to a Telegram chat; and an Ambulance Unit (ESP32 with GPS and SIM800L) that streams its live position — coordinated by a Node.js backend server that ingests events over MQTT, persists them in a SQLite database, computes ambulance-to-accident driving distance and ETA using the Open Source Routing Machine (OSRM), and serves a live Leaflet.js/OpenStreetMap dashboard over a WebSocket connection.

The assembled prototype was evaluated through unit, integration, and end-to-end system testing, confirming that a simulated impact correctly triggers the full alert-to-assignment chain, that redundant alerting (SMS and MQTT) functions independently of one another, and that the nearest-ambulance assignment algorithm correctly identifies the closer of multiple candidate ambulances and returns a realistic driving route and ETA. The thesis concludes with a discussion of the system's advantages, its current limitations (notably its threshold-based, rather than machine-learning-based, crash classifier, and its single-nearest-ambulance rather than fleet-optimal assignment strategy), and a concrete roadmap of future work.

Keywords: Internet of Things, accident detection, MQTT, GPS/GSM, ESP32, OpenStreetMap, OSRM, ambulance dispatch, embedded systems.

TABLE OF CONTENTS

CHAPTER 1	11
1.1 Overview	11
1.2 System Block Diagram and Working Principle	11
1.3 Motivation	14
1.4 Problem Statement	14
1.5 Objectives	15
1.6 Scope and Limitations	16
1.7 Methodology Overview	16
1.8 Organization of the Thesis	17
CHAPTER 2	18
2.1 Introduction	18
2.2 Accelerometer-Based Accident Detection Systems	18
2.3 GPS/GSM Based Vehicle Tracking and Alert Systems	19
2.4 IoT Architectures for Vehicular Safety	19
2.5 Emergency Response and Ambulance Dispatch Systems	20
2.6 Open-Source Mapping, Routing, and OSRM	21
2.7 Research Gap and Motivation for the Proposed System	21
CHAPTER 3	23
3.1 System Overview	23
3.2 Design Requirements	23
3.3 Overall System Block Diagram	24
3.4 Vehicle Unit Design	24
3.5 Camera Module Design	25
3.6 Ambulance Unit Design	26
3.7 Backend Server Design	26
3.8 MQTT Communication Design	27
3.9 Ambulance Assignment Algorithm	28
3.10 Dashboard Design	29
CHAPTER 4	30
4.1 Hardware Components Used	30
4.2 Pin Connection Diagram	30
4.3 Vehicle Unit Firmware Implementation	32
4.4 Camera Module Firmware Implementation	33
4.5 Ambulance Unit Firmware Implementation	34
4.6 Server-Side Implementation	34
4.7 Database Schema Implementation	35
4.8 Dashboard Implementation	36
4.9 Testing Strategy	36
CHAPTER 5	38
5.1 Results Overview	38

5.2 Vehicle Unit Hardware Setup.....	38
5.3 Accident Alert Results.....	39
5.4 Camera Module and Telegram Upload Results.....	39
5.5 Ambulance Tracking and Dashboard Results	40
5.6 Performance Evaluation	41
5.7 Discussion of Results	42
5.8 Advantages of the Proposed System.....	42
5.9 Limitations	43
5.10 Future Scope	43
5.11 Conclusion	44
REFERENCES	45

(Right-click the table above and choose "Update Field" in Microsoft Word to populate page numbers.)

LIST OF FIGURES

Fig. 1.1 System Block Diagram — Accident Detection, Alert & Ambulance Navigation System

Fig. 4.1 Vehicle Unit — Pin Connection Diagram (ESP32-S3)

Fig. 5.1 Assembled Vehicle Unit hardware prototype

Fig. 5.2 Vehicle Unit OLED status display (ARMED state)

Fig. 5.3 Emergency SMS alert received on the configured phone

Fig. 5.4 Accident event received and logged by the backend server

Fig. 5.5 Accident clip received in the configured Telegram chat

Fig. 5.6 Live dashboard showing ambulance and accident markers

Fig. 5.7 Dashboard showing the OSRM-computed route and ETA after assignment

LIST OF TABLES

Table 3.1 MQTT Topic Schema

Table 4.1 Hardware Bill of Materials

Table 4.2 Pin Assignment Summary

Table 4.3 Database Table Summary

Table 5.1 Sample Captured Clip Characteristics

Table 5.2 End-to-End Stage Latency (representative)

Table 5.3 Crash-Detection Test Results (representative)

LIST OF ABBREVIATIONS

Abbreviation	Full Form
IoT	Internet of Things
GPS	Global Positioning System
GSM	Global System for Mobile Communications
GPRS	General Packet Radio Service
MQTT	Message Queuing Telemetry Transport
OSM	OpenStreetMap
OSRM	Open Source Routing Machine
ETA	Estimated Time of Arrival
IMU	Inertial Measurement Unit
MEMS	Micro-Electro-Mechanical Systems
I2C	Inter-Integrated Circuit
UART	Universal Asynchronous Receiver-Transmitter
JSON	JavaScript Object Notation
REST	Representational State Transfer
SD	Secure Digital (memory card)
ML	Machine Learning

CHAPTER 1

INTRODUCTION

1.1 Overview

Road traffic accidents remain one of the leading causes of preventable death worldwide, and a significant share of that mortality is not caused by the collision itself but by the delay between the moment of impact and the arrival of emergency medical help. In many accidents, particularly those occurring on highways, rural roads, or during night hours, there is no witness capable of raising an alarm quickly enough. The victim, if conscious, may be unable to call for help, and if unconscious, no call is made at all until a passer-by happens to notice the wreckage. This gap between impact and intervention, often referred to in emergency medicine as the “golden hour,” is the single largest controllable factor in whether a road traffic victim survives.

This thesis presents the design, implementation, and evaluation of an Internet of Things (IoT) based Accident Detection, Alert and Ambulance Navigation System. The system is built around three cooperating hardware units — a Vehicle Unit that senses the crash itself, a Camera Module that preserves video evidence of the moments surrounding the crash, and an Ambulance Unit that reports its live position — all coordinated by a backend server that automatically determines the nearest available ambulance and computes a real driving route and estimated time of arrival using free, open-source mapping data. The explicit goal of the project is to compress the time between “accident happens” and “the right ambulance is on its way, guided efficiently” down to a matter of seconds, without requiring any human being to make a phone call.

Unlike many published prototypes in this space, the system described here is not a single microcontroller sketch but a complete, deployable architecture: independent firmware projects for each physical unit, a Node.js backend that persists state and performs routing, and a live web dashboard for monitoring. Every design decision in the chapters that follow — the choice of communication protocol, the crash-detection thresholds, the routing engine, the database schema — was made with real-world deployability and cost in mind, using exclusively free and open-source components.

1.2 System Block Diagram and Working Principle

Figure 1.1 presents the complete system block diagram, showing every hardware component of the three field units, the independent wireless links between them, and the backend/dashboard infrastructure they report to. This diagram is introduced here, in the opening chapter, because the remainder of the thesis repeatedly refers back to it while explaining individual subsystems in detail — Chapter 3 revisits it component-by-component as part of the system design, and Chapter 4 drills into the implementation of each block.

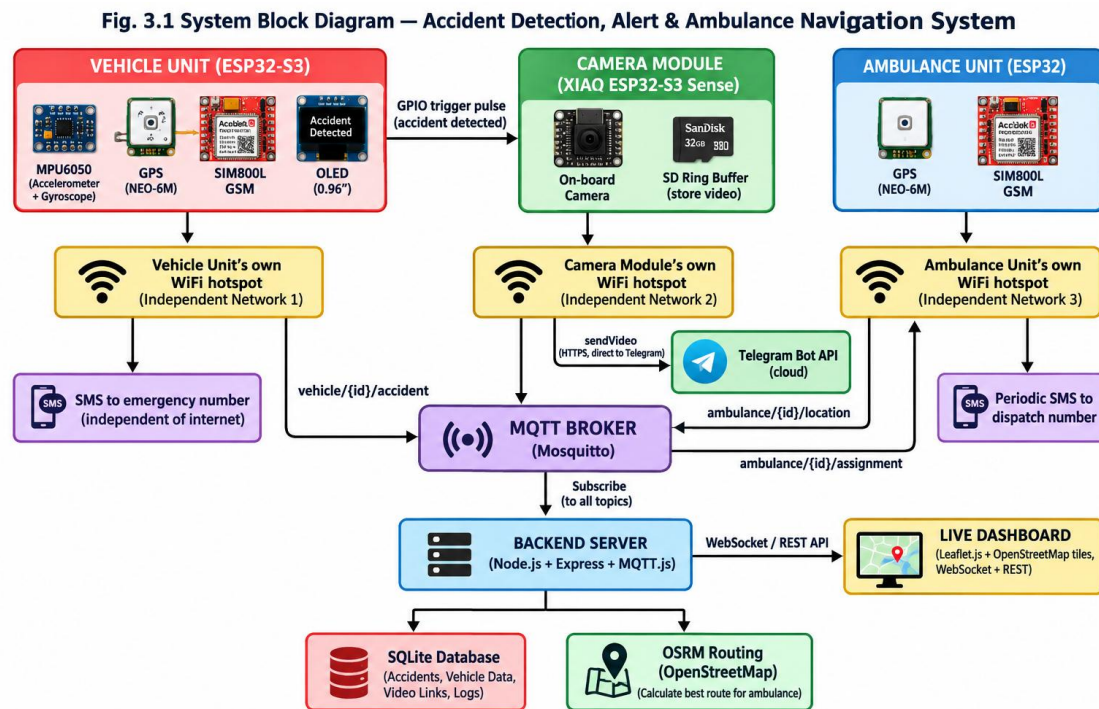


Fig. 1.1 System Block Diagram — Accident Detection, Alert & Ambulance Navigation System

Reading the diagram left to right and top to bottom: the Vehicle Unit (top left, red) carries four sensing/output peripherals — the MPU6050 accelerometer and gyroscope, the NEO-6M GPS module, the SIM800L GSM module, and a small OLED display that shows “Accident Detected” once a crash is confirmed. The moment the Vehicle Unit's onboard logic confirms an accident, it does two things simultaneously: it fires a GPIO trigger pulse across a direct wired connection to the Camera Module (top centre, green), and it begins its own alerting sequence. The Camera Module, built on a XIAO ESP32-S3 Sense, holds a continuously updating SD ring buffer of recent video; the trigger pulse tells it to stop overwriting that buffer and permanently save the clip spanning the accident.

Each of the three field units — Vehicle Unit, Camera Module, and Ambulance Unit (top right, blue) — connects to its own independent WiFi hotspot, labelled in the diagram as Independent Network 1, 2, and 3 respectively. This is a deliberate architectural choice, not an implementation detail: none of the three networks needs to be the same network, and none of the three units depends on another unit's network being available. From its own hotspot, the Vehicle Unit sends the accident payload (topic `vehicle/{id}/accident`) to the MQTT Broker (centre, purple), while independently also sending an SMS with the accident location directly to a configured emergency number over the SIM800L GSM module — a path that works even with zero WiFi/internet connectivity at all, since SMS travels over the 2G cellular network rather than the data connection.

The Camera Module, once it has finished saving the clip, uploads it directly to the Telegram Bot API (a cloud service, shown centre-right in green) over its own WiFi connection — this path does not go through the MQTT broker or the backend server at all, keeping video delivery decoupled from the rest of the system. The Ambulance Unit continuously publishes its live GPS position to the broker (topic `ambulance/{id}/location`) and separately sends its own periodic SMS location update to a dispatch number, exactly mirroring the Vehicle Unit's dual-channel pattern.

The Backend Server (bottom centre, blue) subscribes to every topic on the broker. When it receives an accident event, it stores the event in the SQLite Database, calls the OSRM Routing engine to compute the driving distance and ETA from the nearest available ambulance, and publishes the result back to that ambulance's own assignment topic (`ambulance/{id}/assignment`) — the one arrow in the diagram that flows from the broker back out to a field unit. Every state change along the way — a new accident, a moved ambulance, a new assignment — is simultaneously pushed to the Live Dashboard (bottom right, yellow) over a WebSocket connection, so an operator watching the Leaflet/OpenStreetMap map sees the whole sequence unfold in real time, typically within a second or two of the underlying event.

This single diagram therefore encodes the three properties that distinguish this system's architecture from a simpler single-network design, each elaborated further in later chapters: (i) independent connectivity for every field unit (Section 3.2, requirement R1), (ii) redundant, connectivity-independent alerting via SMS alongside MQTT (requirement R2), and (iii) a strict separation between the data path (MQTT, for structured events the server

must act on) and the media path (direct-to-Telegram, for the video payload the server does not need to process).

1.3 Motivation

The motivation for this work comes from three converging observations. First, road accident statistics consistently show that a large proportion of fatalities occur not at the scene but during transport to hospital, or while waiting for transport to begin — a delay problem, not purely a medical one. Second, the components needed to solve this delay problem have, over the last decade, become inexpensive enough to place inside every vehicle: micro-electromechanical accelerometers cost a few tens of rupees, GPS modules are commodity items, and cellular/WiFi connectivity is nearly universal. Third, the mapping and routing infrastructure required to direct an ambulance efficiently — OpenStreetMap and the Open Source Routing Machine (OSRM) — is now mature, free, and self-hostable, removing what used to be the single largest recurring cost (paid mapping APIs) from any such system.

Existing commercial accident-alert products typically fall into one of two categories: expensive, proprietary in-vehicle telematics units sold as an add-on by insurers or automakers, or simple GPS trackers that report location but perform no crash detection and no ambulance coordination at all. Neither category addresses the complete chain of events an accident actually requires — detect, alert, record evidence, locate the nearest responder, and route that responder efficiently. This project was motivated by the belief that a system covering that entire chain could be built from freely available hardware modules and open-source software, and documented clearly enough that it could be replicated, extended, or adapted by future students and researchers.

A secondary motivation is pedagogical: the project deliberately spans embedded firmware, real-time sensor processing, wireless communication protocols, backend web services, geographic routing algorithms, and live data visualization. It therefore serves as a comprehensive case study in end-to-end IoT system design, rather than a narrow demonstration of any single technology.

1.4 Problem Statement

Design and implement a low-cost, independently networked IoT system that (a) automatically detects a vehicular accident using onboard motion sensing, (b) alerts a

configured emergency contact and a backend server within seconds of detection through multiple redundant channels, (c) captures and preserves video evidence spanning the moments before and after the impact, (d) tracks the live position of one or more ambulances, and (e) automatically identifies the nearest available ambulance to a reported accident and computes the driving distance and estimated time of arrival between them using open, non-proprietary mapping data — all while ensuring that no single component of the system depends on another component's network connection for its own core function.

1.5 Objectives

- To design a threshold-based crash-detection algorithm using a 6-axis accelerometer/gyroscope that reliably distinguishes a high-impact event from normal driving vibration, while remaining structured so that a machine-learning classifier can later replace the threshold logic without redesigning the surrounding firmware.
- To implement a Vehicle Unit that, on detecting a probable accident, gives the occupant a short cancellable countdown (to reject false positives), then sounds an audible alarm, sends an SMS containing a map link to a configured emergency number, and publishes a structured accident event to a backend server.
- To implement an independent Camera Module that continuously buffers video from an onboard camera and, on receiving a trigger signal from the Vehicle Unit, permanently saves a clip spanning the pre-impact and post-impact period to local storage and uploads it to a Telegram chat for remote viewing.
- To implement an Ambulance Unit that continuously reports its GPS location to the backend server and can receive an assignment notification identifying an accident it has been dispatched to.
- To design and implement a backend server that ingests events over the MQTT publish-subscribe protocol, persists accident and ambulance-location history, automatically assigns the nearest available ambulance to a new accident, and computes real driving distance and ETA using the OpenStreetMap-based OSRM routing engine — explicitly avoiding any Google Maps or other paid mapping API.

- To provide a live, map-based dashboard, built with Leaflet.js and OpenStreetMap tiles, showing all ambulance positions, active accident locations, and the computed route and ETA between an assigned ambulance and its accident, updating in real time.
- To ensure every physical unit in the system has its own independent internet connectivity, so that no unit's core safety function is blocked by another unit's network being unavailable.

1.6 Scope and Limitations

The scope of this project covers the complete signal chain from crash detection to ambulance dispatch visualization, implemented and tested at prototype scale using development boards (ESP32-S3, ESP32, and the Seeed XIAO ESP32-S3 Sense) rather than automotive-grade hardware. The crash-detection algorithm implemented for this thesis is threshold-based rather than machine-learning-based; this was a deliberate scope decision, since a robust ML classifier requires a labelled crash dataset that was outside the resources available for this project, but the firmware is explicitly structured so that a TensorFlow Lite Micro model can be substituted for the threshold function without any other code changes.

Similarly, the routing component relies on the public OSRM demonstration server during development and testing; a production deployment would self-host OSRM against a regional OpenStreetMap extract, which is documented in this thesis as a deployment recommendation but was not itself load-tested at scale. The ambulance-assignment algorithm implemented is a single-nearest-available-ambulance strategy; more sophisticated multi-ambulance, multi-accident optimal assignment (an assignment-problem formulation) is identified as future work in Chapter 5.

The system does not attempt to replace or interface with existing national emergency-dispatch (e.g., 108/112) telephony infrastructure; it operates as a parallel, self-contained alerting and coordination channel that a fleet operator, hospital, or private ambulance service could deploy independently.

1.7 Methodology Overview

The project followed an iterative, component-first methodology. Each of the four independent subsystems — Vehicle Unit, Camera Module, Ambulance Unit, and Backend Server — was designed against an explicit interface contract (a defined MQTT topic and JSON payload schema, or a defined GPIO trigger signal) before implementation began, so

that the subsystems could be built, modified, and tested independently of one another. The backend server and its live dashboard were implemented first, since every other subsystem's correctness is ultimately verified by observing its effect there. Firmware for each embedded unit was then implemented against that already-working backend, using modular per-peripheral source files (GPS, GSM, IMU, display, MQTT client) so that any one peripheral driver could be replaced or debugged without touching the others.

Verification was carried out at three levels: unit-level (each firmware module tested against its own hardware peripheral, and each backend module covered by direct invocation), integration-level (the full MQTT message flow from a simulated accident event through to a dashboard update), and system-level (an end-to-end walkthrough of the accident-to-assignment-to-dashboard chain, described with results captured in Chapter 5).

1.8 Organization of the Thesis

The remainder of this thesis is organized into four further chapters. Chapter 2 surveys existing work in accident detection, GPS/GSM vehicle alert systems, IoT architectures for vehicular safety, ambulance dispatch systems, and open-source routing, and identifies the specific gap this project addresses. Chapter 3 presents the overall system design and architecture, including the system block diagram, the design of each subsystem, the MQTT communication schema, and the ambulance-assignment algorithm. Chapter 4 describes the concrete implementation of each subsystem — hardware wiring and pin assignment, firmware logic, and backend/server code — in enough detail to reproduce the system. Chapter 5 presents the results obtained from testing the assembled system, evaluates its performance, discusses its advantages and limitations, proposes future extensions, and concludes the thesis.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

This chapter surveys prior work relevant to the four technical pillars of the proposed system: accelerometer-based accident detection, GPS/GSM based vehicle tracking and alerting, IoT architectures applied to vehicular safety, emergency-response and ambulance-dispatch systems, and open-source mapping/routing infrastructure. Each section summarizes the general approach taken in the literature, notes its strengths, and identifies the specific limitation that motivates a design choice made later in this thesis. The chapter closes by consolidating these observations into an explicit research gap.

2.2 Accelerometer-Based Accident Detection Systems

The earliest and most widely replicated approach to automatic crash detection uses a single- or multi-axis accelerometer to look for a sudden, large-magnitude deceleration spike consistent with an impact, typically compared against a fixed g-force threshold. This approach is attractive because it requires only a single low-cost sensor and no network connectivity to make its detection decision, and it has been the basis of numerous undergraduate and early-career research prototypes combining an accelerometer with a GSM module for SMS alerting.

A recurring criticism in the literature is the high false-positive rate of a pure accelerometer threshold: potholes, speed breakers, hard braking, and even slamming a car door can produce accelerations in the same range as a low-speed collision. Later work addressed this by combining the accelerometer with a gyroscope to also detect the abnormal rotational motion (rollover, spin) that typically accompanies a genuine collision but not routine braking, and by introducing a short confirmation window — either a fixed delay before alerting, or a manual cancel button — to let a false positive be dismissed by the occupant. This combined accelerometer-plus-gyroscope, threshold-plus-confirmation-window pattern is the specific design this thesis adopts for its Vehicle Unit, implemented with an MPU6050 six-axis sensor and a ten-second cancellable countdown, as described in Chapter 3.

More recent research has moved toward machine-learning classifiers — typically small neural networks or decision trees trained on labelled crash/non-crash accelerometer traces

— running either on a smartphone or on a microcontroller via frameworks such as TensorFlow Lite Micro. These approaches report materially lower false-positive rates than fixed thresholds but require a labelled crash dataset that is difficult and ethically sensitive to collect at scale, which is the reason this thesis does not attempt a full ML classifier, while still structuring its firmware so that one can be substituted later.

2.3 GPS/GSM Based Vehicle Tracking and Alert Systems

A large body of prior work pairs a GPS receiver with a GSM module to build vehicle tracking or emergency-SMS systems: the GPS module supplies coordinates, and the GSM module either transmits them periodically to a monitoring number, or sends them once as part of an emergency alert triggered by some external event (a panic button, or an accident sensor). This SMS-based channel has the significant practical advantage of not requiring mobile data or WiFi at all — a plain 2G SIM connection is sufficient — which makes it valuable as a fallback channel even in systems that primarily use data connectivity, and is why this thesis retains SMS via a SIM800L module as an always-available alert path alongside its primary MQTT-over-WiFi channel.

The limitation consistently reported for pure SMS/GSM systems is that they are a one-way, human-in-the-loop channel: an SMS informs a single phone number, but nothing in the system itself decides what to do next — no automatic matching of the reported location to a responder, no computed distance or route, no persistent record. Systems in this category are alerting systems, not coordination systems, which is precisely the gap the present work's backend server, database, and assignment algorithm are designed to close.

2.4 IoT Architectures for Vehicular Safety

As internet-connected microcontrollers (particularly the ESP8266/ESP32 family) became inexpensive, a second generation of vehicle-safety prototypes replaced or supplemented SMS alerting with internet-based publish-subscribe messaging, most commonly MQTT, sending structured JSON event payloads to a cloud or self-hosted broker rather than a plain-text SMS. This shift enables a genuinely automated backend to react to an event — store it, analyze it, forward it to multiple recipients, or trigger further logic — in a way that an SMS message cannot support directly.

The literature in this area typically treats “the cloud” as a single monolithic destination and gives limited attention to what happens when a vehicle briefly loses its data connection at

the exact moment of an accident — arguably the most safety-critical failure mode in the entire system, since a vehicle is statistically more likely to be somewhere with poor coverage (a rural road, an underpass, a remote highway) at the time of a serious accident than at a random moment. This thesis addresses that gap directly by queuing failed MQTT publishes for retry and by treating the SMS channel as an always-on, connectivity-independent backup for the accident alert specifically, rather than as the primary channel.

A separate strand of IoT-safety literature addresses multi-device coordination — systems where several independently networked devices must cooperate without assuming a shared local network. This thesis's explicit requirement that the Vehicle Unit, Camera Module, and Ambulance Unit each maintain their own independent WiFi connectivity, coordinating only through a shared backend reachable over the public internet, follows this pattern and is discussed further in Chapter 3.

2.5 Emergency Response and Ambulance Dispatch Systems

Research on computer-aided ambulance dispatch generally falls into two areas: (a) optimal fleet-assignment algorithms, which model dispatch as a variant of the assignment problem or as a queueing/coverage optimization over a region, and (b) live-tracking dashboards that visualize ambulance and incident locations for a human dispatcher. Fleet-assignment research tends to be evaluated in simulation against historical incident data from large metropolitan EMS systems, and typically assumes accurate, low-latency positional data for every vehicle in the fleet as a precondition — a precondition this thesis's Ambulance Unit is specifically designed to satisfy by publishing its GPS position over MQTT at a short, configurable interval.

Live-tracking dashboard research has increasingly moved away from proprietary GIS platforms toward open web-mapping stacks (Leaflet.js or OpenLayers rendering OpenStreetMap tiles), both for cost reasons and because open tile servers can be self-hosted for full data sovereignty — a consideration directly relevant to any system handling the location of medical emergencies. This thesis's dashboard follows that same open-stack pattern.

What is comparatively under-addressed in the dispatch literature is the nearest-first, single-assignment case relevant to a small private fleet (as opposed to a citywide public EMS system): given one new accident and a handful of ambulances, quickly identify the nearest

available one and give it a real driving route, without the computational overhead of a full fleet-optimization solver. This is the specific, narrower problem this thesis's assignment algorithm (Chapter 3, Section 3.9) is designed for.

2.6 Open-Source Mapping, Routing, and OSRM

OpenStreetMap (OSM) is a collaboratively maintained, freely licensed geographic database covering road networks worldwide, and has become the standard free alternative to proprietary map data for research and small-scale production systems. Multiple routing engines consume OSM data to compute driving, walking, or cycling routes; the Open Source Routing Machine (OSRM) is among the most widely used, offering sub-second route computation over large road networks through a pre-processed contraction-hierarchy data structure, and exposing a simple HTTP API that returns distance, duration, and route geometry for a given origin-destination pair.

The literature on OSRM deployment consistently notes the distinction between the public OSRM demonstration server — free, unauthenticated, but explicitly rate-limited and unsuitable for continuous production use — and a self-hosted OSRM instance built from a regional OSM extract, which requires a one-time data-preparation step but then imposes no per-request cost or external dependency at all. This thesis uses the public demo server for development and documents self-hosting as the recommended production path, consistent with this established best practice.

2.7 Research Gap and Motivation for the Proposed System

Reviewing the five areas above together reveals that no single strand of prior work addresses the complete chain from sensor-level crash detection through to a routed ambulance dispatch: accelerometer-based detection research stops at the detection decision; GPS/GSM alert research stops at notifying a single phone number; IoT-vehicular-safety research generally treats the destination as an undifferentiated cloud backend without a coordination layer; ambulance-dispatch research assumes incident and vehicle location data simply exists, without addressing how it is generated at the vehicle; and open-mapping/routing research is a general-purpose tool, not tied to any specific emergency-response use case.

The system presented in this thesis is positioned specifically to close this gap: it is an end-to-end pipeline in which the crash-detection decision made by an MPU6050-equipped

Vehicle Unit directly triggers a chain of events — SMS alert, camera evidence capture, MQTT accident event, server-side nearest-ambulance assignment, OSRM-computed driving route, and live dashboard update — implemented entirely with low-cost commodity hardware and free, self-hostable, open-source software, with no proprietary mapping API anywhere in the chain, and with each physical unit maintaining independent network connectivity so that the failure of one unit's connection cannot block another unit's core safety function.

CHAPTER 3

SYSTEM DESIGN AND ARCHITECTURE

3.1 System Overview

The proposed system is composed of four independently deployable subsystems that communicate through well-defined interfaces rather than a shared codebase or shared network. The Vehicle Unit continuously monitors motion and position, decides locally whether an accident has occurred, and reports that decision over three separate channels (buzzer, SMS, MQTT). The Camera Module, physically co-located with the Vehicle Unit but running entirely separate firmware, maintains a rolling video buffer and permanently saves a clip when it receives a simple electrical trigger from the Vehicle Unit. The Ambulance Unit continuously reports its own position. The Backend Server is the only component that has visibility of the whole system: it receives events from all vehicle and ambulance units, persists them, computes ambulance-to-accident assignments and routes, and serves a live dashboard.

This section-by-subsystem decomposition was a deliberate architectural choice, justified in Section 3.2 below, and is what allows each subsystem to be designed, implemented, tested, and even physically deployed on a different vehicle or timeline without the others being affected.

3.2 Design Requirements

The following requirements were established before implementation began and shaped every subsequent design decision in this chapter.

- R1 — Independent connectivity: no subsystem's core safety function may depend on another subsystem's network connection being available.
- R2 — Redundant alerting: the accident alert must reach a human being (via SMS) even if no data/internet connectivity is available at all.
- R3 — False-positive tolerance: the occupant must be able to cancel an alert before it is sent, to avoid nuisance dispatch for non-accident events (potholes, hard braking).
- R4 — No proprietary map dependency: all mapping, routing, and geocoding must use OpenStreetMap-derived, non-paid data sources.

- R5 — Low cost, replicable hardware: every component must be a commodity module available from standard electronics suppliers, avoiding automotive-grade or custom silicon.
- R6 — Extensibility: the crash-detection logic and the ambulance-assignment logic must each be isolated behind a clear interface so they can later be replaced (with a machine-learning classifier, or a fleet-optimization solver, respectively) without a system redesign.

3.3 Overall System Block Diagram

The complete system architecture was already presented as Figure 1.1 in Chapter 1, Section 1.2, and is not repeated here; this section instead revisits it at the component level. The three field-deployed units (Vehicle Unit, Camera Module, Ambulance Unit) each maintain their own independent WiFi hotspot connection and communicate outward to a shared MQTT broker; the Vehicle Unit and Ambulance Unit additionally have an SMS path via their own SIM800L GSM module that does not depend on any data connection at all. The Camera Module's video upload path (Telegram Bot API over HTTPS) is likewise independent of the MQTT path. The Backend Server subscribes to the broker, persists data to a SQLite database, calls the OSRM routing engine for distance/ETA calculation, and pushes live updates to the Leaflet-based dashboard over a WebSocket connection.

A single physical link exists between two field units: a GPIO trigger pulse from the Vehicle Unit to the Camera Module, asserted for approximately 200 milliseconds when an accident alert is fired. This is intentionally the only coupling between any two field units in the entire system — every other interaction happens indirectly, through the backend server, over each unit's own independent internet connection.

3.4 Vehicle Unit Design

The Vehicle Unit is built around an ESP32-S3 microcontroller, chosen for its dual-core processor (allowing sensor polling, display updates, and network I/O to proceed without blocking one another), ample GPIO count for the five attached peripherals, and native WiFi support. Its design decomposes into five concerns, each isolated into its own firmware module: GPS acquisition (`gps.cpp`), GSM/SMS handling (`gsm.cpp`), inertial sensing and crash detection (`mpu6050.cpp`), status display (`oled.cpp`), and MQTT communication

(`mqtt_client.cpp`), coordinated by a small state machine in `main.cpp` with three states — ARMED, COUNTDOWN, and ALERTING.

In the ARMED state, the unit polls the MPU6050 every 50 milliseconds and evaluates a threshold function (detailed in Section 3.4.1) against each reading. A positive detection transitions the unit to COUNTDOWN, in which the buzzer sounds and a configurable window (default 10 seconds) begins, during which a physical button press returns the unit directly to ARMED, discarding the event as a false positive. If the countdown expires without cancellation, the unit transitions to ALERTING: it fires the camera-trigger GPIO pulse, sends the SMS alert, publishes the MQTT accident event, and then returns to ARMED to continue monitoring.

3.4.1 Crash-Detection Algorithm. The detection function computes two scalar quantities from each accelerometer/gyroscope reading: the magnitude of the acceleration vector expressed in units of gravitational acceleration (g), and the magnitude of the angular velocity vector in degrees per second. A reading is classified as a possible impact if either the acceleration magnitude deviates from the resting value of 1g by more than a configurable threshold (default 3.5g), or the angular velocity magnitude exceeds a configurable threshold (default 250°/s) — the latter condition specifically to catch rollover or spin events that a pure acceleration threshold can miss. Both thresholds are compile-time constants in the unit's configuration file, intended to be tuned empirically for a specific vehicle and sensor mounting.

3.5 Camera Module Design

The Camera Module runs on a Seeed XIAO ESP32-S3 Sense, chosen specifically because it integrates a camera, a microSD slot, and 8 MB of PSRAM on a single small board — the PSRAM being the critical resource for buffering video frames, discussed further in Chapter 4. Because the module has no hardware video encoder, the design represents a video clip as Motion-JPEG: a sequence of independently compressed JPEG frames, each captured directly by the camera driver, concatenated into an AVI container without any re-encoding step.

The module maintains a fixed-size ring buffer of the most recent pre-trigger frames in PSRAM. On receiving the trigger signal from the Vehicle Unit, it flushes this buffer (oldest frame first) into a new AVI file on the SD card, then continues capturing and appending

fresh frames directly to that file for a configured post-trigger duration, before finalizing the file's container headers. This design means the module never needs to hold the full clip in memory at once — only the fixed-size pre-trigger window — which is what makes a fifty-second clip feasible on a device with a few megabytes of usable PSRAM. Once the file is finalized, it is uploaded to a configured Telegram chat via the Telegram Bot API's `sendVideo` method, streamed directly from the SD card; if no WiFi connection is available at that moment, the file remains on the SD card and is retried automatically the next time the module regains connectivity or is rebooted.

3.6 Ambulance Unit Design

The Ambulance Unit is deliberately simpler than the Vehicle Unit: it has no crash-detection responsibility, so it omits the IMU and display, retaining only a GPS receiver and a SIM800L GSM module alongside its ESP32 microcontroller and WiFi connection. Its firmware continuously reads GPS and publishes a location payload over MQTT at a configurable interval (default seven seconds, within the five-to-ten-second range specified in the project requirements), and separately sends a plain-text SMS with its location to a configured dispatch number on a longer timer (default five minutes), independent of the MQTT channel. It subscribes to an assignment topic scoped to its own device ID, so that when the backend server assigns it to a specific accident, it can log that assignment and reflect an “assigned” status in its next location publish — this status is round-tripped back to the server purely as a display/logging convenience, since the server itself is the authority on assignment state.

3.7 Backend Server Design

The backend server is implemented in Node.js using Express for its HTTP API, the MQTT.js library for broker connectivity, Node's built-in `node:sqlite` module for persistence, and the `ws` library for pushing live updates to connected dashboard clients. Node.js/JavaScript was chosen over an equivalent Python (FastAPI + `paho-mqtt`) implementation for this project specifically because it allows the MQTT client, the REST API, the WebSocket server, and the static dashboard files to all run inside a single process with a single dependency tree and no separate build step — a meaningful simplicity advantage for a project-scale system that must remain easy to deploy and reason about as a whole, while Python's ecosystem was judged to offer no corresponding advantage for this particular workload.

The server is organized into five layers: an MQTT layer (connection management and message routing, `mqtt/client.js` and `mqtt/handlers.js`) that receives raw broker messages and dispatches them by topic; a persistence layer (`db/index.js` and `db/models.js`) exposing typed query functions over four tables (accidents, ambulances, ambulance_locations, assignments); a routing layer (`routing/osrm.js`) that wraps the OSRM HTTP API with a haversine-distance fallback if the routing service is unreachable; a service layer (`services/assignment.js`, `services/telegram.js`) implementing the ambulance-assignment algorithm and optional Telegram notifications; and a presentation layer (`routes/*.js`, `ws/broadcast.js`) exposing REST endpoints and broadcasting live events to the dashboard.

3.8 MQTT Communication Design

MQTT (Message Queuing Telemetry Transport) was selected as the primary data-channel protocol because it is a lightweight, publish-subscribe protocol specifically designed for constrained devices and unreliable networks — both properties directly relevant to microcontrollers on a mobile WiFi hotspot connection. Table 3.1 defines the four topics used by the system and their direction of flow.

Table 3.1 MQTT Topic Schema

Topic	Direction	Purpose
vehicle/{device_id}/accident	Vehicle → Server	Accident event payload (QoS 1)
vehicle/{device_id}/status	Vehicle → Server	Optional periodic heartbeat
ambulance/{device_id}/location	Ambulance → Server	Live GPS location, every 5–10 s
ambulance/{device_id}/assignment	Server → Ambulance	Assignment result (accident id, distance, ETA)

The accident-event and ambulance-location payloads are both compact JSON objects, shown in Listing 3.1 and Listing 3.2 respectively; the assignment payload sent back to an ambulance is shown in Listing 3.3. Keeping every payload a small, flat JSON object (rather than a nested or binary structure) was a deliberate choice to keep firmware-side serialization simple using the ArduinoJson library, and to keep the schema trivially human-readable for debugging with a plain MQTT client during development.

```
{
  "device_id": "VEH-001",
  "timestamp": "2026-09-23T10:15:30Z",
```

```

    "lat": 12.9716,
    "lon": 77.5946,
    "event_type": "accident",
    "severity": "high",
    "impact_g": 4.2,
    "video_ref": null
  }

```

```

{
  "device_id": "AMB-001",
  "timestamp": "2026-09-23T10:16:00Z",
  "lat": 12.9700,
  "lon": 77.5900,
  "speed_kmh": 40.5,
  "status": "available"
}

```

```

{
  "accident_id": "5e1b3f2a-...",
  "lat": 12.9716,
  "lon": 77.5946,
  "distance_km": 3.2,
  "eta_minutes": 6.4
}

```

3.9 Ambulance Assignment Algorithm

When a new accident event is received, the server must select an ambulance to dispatch. A naive approach would call the OSRM routing API once per candidate ambulance to find the one with the shortest actual driving distance; this was rejected because it does not scale gracefully against the public OSRM demonstration server's rate limits, and because most of those route calculations would be wasted on candidates far from the eventual answer. The algorithm instead uses a two-stage filter, described in Figure 3.2 and summarized below.

Stage 1 (coarse filter): the server retrieves every ambulance currently marked “available” with a known GPS position, computes the straight-line (haversine) distance from each to the accident, discards any ambulance beyond a configurable search radius (default 25 km), and sorts the remainder by that straight-line distance.

Stage 2 (confirmation): the single closest candidate from Stage 1 is confirmed with one OSRM route request, which returns the actual driving distance and estimated time of arrival along real roads, along with the route geometry used to draw the path on the dashboard.

The accident record is then updated to “assigned” with a reference to the chosen ambulance, the ambulance's own status is updated to “assigned,” an assignment record is persisted for audit/history purposes, and the assignment (including the computed distance and ETA) is published back to the ambulance's own MQTT assignment topic and broadcast to every connected dashboard client. If no ambulance is available within the search radius, the accident is left in the “active, unassigned” state and a warning is logged; this scenario, and its handling, is discussed further as a limitation in Chapter 5.

3.10 Dashboard Design

The live dashboard is a static single-page web application, served directly by the backend's Express instance, built with the Leaflet.js mapping library rendering OpenStreetMap tiles — chosen over any Google Maps based alternative to satisfy the project's non-proprietary-mapping requirement (R4) and because Leaflet's plugin-free core is sufficient for the marker, popup, and polyline primitives the dashboard needs. On load, the page fetches the current set of active accidents and known ambulance positions over the REST API, then opens a WebSocket connection to receive live updates for three event types — a new accident being created, an ambulance's location changing, and a new assignment being made (which also carries the OSRM route geometry, drawn as a polyline connecting the assigned ambulance to its accident). This design keeps the dashboard's own logic minimal: it holds no business logic of its own and simply renders whatever state the server pushes to it.

CHAPTER 4

IMPLEMENTATION

4.1 Hardware Components Used

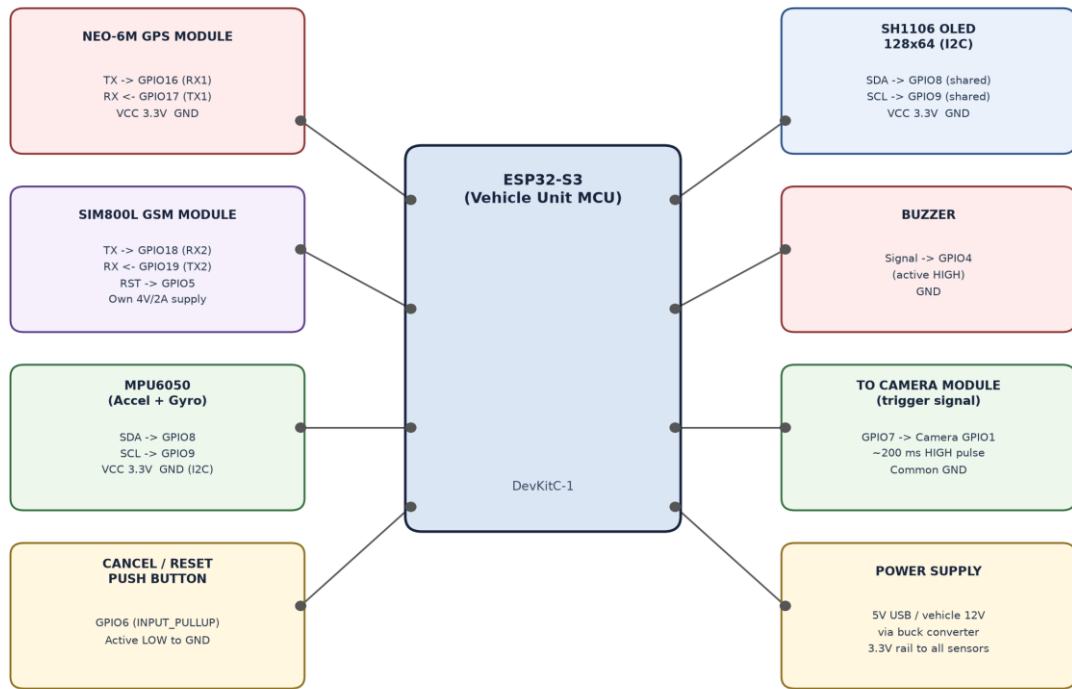
Table 4.1 lists the complete hardware bill of materials used across the three field units of the prototype.

Table 4.1 Hardware Bill of Materials

Unit	Component	Interface
Vehicle Unit	ESP32-S3 (DevKitC-1)	Main controller
Vehicle Unit	NEO-6M GPS module	UART
Vehicle Unit	SIM800L GSM module	UART + dedicated power
Vehicle Unit	MPU6050 accelerometer/gyroscope	I2C
Vehicle Unit	SH1106 128×64 OLED display	I2C
Vehicle Unit	Buzzer, push button	Digital GPIO
Camera Module	Seeed XIAO ESP32-S3 Sense	Main controller + camera + SD
Ambulance Unit	ESP32 (generic DevKit)	Main controller
Ambulance Unit	GPS module	UART
Ambulance Unit	SIM800L GSM module	UART + dedicated power

4.2 Pin Connection Diagram

Figure 4.1 shows the complete pin connection diagram for the Vehicle Unit, the most peripheral-dense of the three field units. The GPS and GSM modules each communicate over a dedicated hardware UART (UART1 and UART2 respectively) so that neither blocks the other or the USB/debug serial console; the MPU6050 and the SH1106 OLED share a single I2C bus, which is acceptable because they are read/written from the same core at different, non-overlapping points in the main loop. The camera-trigger output and the cancel button are simple digital GPIOs, and the buzzer is driven directly from a GPIO through its own transistor driver stage (omitted from the logical diagram below for clarity, but required in the physical build since the ESP32-S3's GPIO cannot source the current a typical piezo buzzer module draws at full volume).

Fig. 4.1 Vehicle Unit — Pin Connection Diagram (ESP32-S3)

Note: exact GPIO numbers are configurable in include/config.h and must be verified against the physical board revision before wiring.

Fig. 4.1 Vehicle Unit — Pin Connection Diagram (ESP32-S3)

Table 4.2 tabulates the same pin assignments in text form for quick reference during wiring, together with the equivalent assignments used on the Ambulance Unit, which omits the MPU6050, OLED, buzzer, cancel button, and camera-trigger output entirely, retaining only the GPS and GSM UARTs.

Table 4.2 Pin Assignment Summary

Signal	Vehicle Unit GPIO	Ambulance Unit GPIO
GPS RX / TX	16 / 17	16 / 17
GSM RX / TX	18 / 19	18 / 19
GSM reset	5	5
MPU6050 SDA / SCL	8 / 9	—
OLED SDA / SCL	8 / 9	—
Buzzer	4	—
Cancel button	6	—
Camera trigger out	7	—

The SIM800L GSM module deserves a specific implementation note: unlike the other peripherals, it cannot be powered from the ESP32's onboard 3.3V regulator, since its transmit current can spike to approximately 2A during a GSM transmission burst, which a typical onboard regulator cannot supply without browning out the microcontroller itself. Both the Vehicle Unit and Ambulance Unit builds therefore use a dedicated 3.7–4.2V, 2A-capable supply (either a LiPo cell with a protection circuit, or a dedicated buck converter sized for the peak current) feeding the SIM800L independently of the ESP32's own power input.

4.3 Vehicle Unit Firmware Implementation

The Vehicle Unit firmware is organized as a PlatformIO project targeting the ESP32-S3 DevKitC-1 board definition, with library dependencies on TinyGPSPlus (NMEA sentence parsing), TinyGSM (SIM800L AT-command handling), PubSubClient (MQTT), ArduinoJson (payload serialization), Adafruit MPU6050 and Adafruit Unified Sensor (IMU driver), and U8g2 (SH1106 display driver). Listing 4.1 shows the core of the crash-detection function implemented in `mpu6050.cpp`, which is invoked once per IMU reading from the main loop and returns a boolean decision without any knowledge of what happens after a positive detection — that separation of concerns is what allows the function to be swapped for a machine-learning classifier later without touching `main.cpp`.

```
bool mpuDetectCrash(const ImuReading &r) {
    // Sudden high-G deviation from resting 1g, OR abnormal rotation rate.
    float deviationG = fabsf(r.accelMagG - 1.0f);
    return deviationG >= IMPACT_G_THRESHOLD || r.gyroMagDps >= GYRO_DPS_THRESHOLD;
}
```

The MQTT client module (`mqtt_client.cpp`) wraps `PubSubClient` with an automatic reconnect loop and a small fixed-size retry queue: any publish attempted while the broker connection is down is appended to the queue (dropping the oldest entry if the queue is full, on the reasoning that a stale queued event is less useful than a fresh one) and flushed automatically once connectivity is restored, on every iteration of the main loop. This ensures that a transient WiFi drop at the exact moment of an accident does not silently discard the accident event — it is instead sent as soon as the connection recovers, while the independent SMS channel (Section 4.3.1) has already alerted the emergency contact in the meantime.

4.3.1 SMS Alerting. On entering the ALERTING state, the firmware constructs a plain-text SMS containing the device ID and an OpenStreetMap coordinate link of the form `https://www.openstreetmap.org/?mlat={lat}&mlon={lon}`, which opens directly to the accident location in any browser without requiring the recipient to have a specific mapping application installed, and sends it via the SIM800L's AT-command SMS interface, independent of and in parallel with the MQTT publish.

4.4 Camera Module Firmware Implementation

The Camera Module firmware targets the Seeed XIAO ESP32-S3 Sense board definition and uses the ESP32 Arduino core's bundled `esp_camera` driver, configured for JPEG output at QVGA resolution (320×240 pixels) and a capture rate of five frames per second — a resolution/frame-rate combination chosen specifically to keep the memory budget for the pre-trigger ring buffer within the module's available PSRAM, discussed quantitatively in Section 4.4.1.

4.4.1 Memory Budget for Continuous Buffering. At QVGA resolution and JPEG quality setting 12, empirical frame sizes for this class of sensor typically fall in the 8–15 KB range. Buffering twenty seconds of pre-trigger video at five frames per second requires one hundred stored frames, or roughly 0.8–1.5 MB; the post-trigger thirty seconds of video (a further one hundred and fifty frames) is written directly to the SD card as it is captured rather than held in RAM, so it does not add to the PSRAM budget. This leaves the pre-trigger ring buffer, at under 1.5 MB, comfortably within the module's 8 MB of PSRAM alongside the camera driver's own double frame buffer and the rest of the application's heap usage — the specific design decision (buffer only the pre-trigger window in RAM; stream the post-trigger window straight to storage) that makes a fifty-second clip feasible at all on this hardware.

4.4.2 AVI Container Writer. Since the module has no hardware video encoder, Listing 4.2 shows the structure of the custom Motion-JPEG AVI writer implemented for this project: it writes the RIFF/AVI header structure with placeholder size fields as each JPEG frame chunk is appended, then backfills the RIFF size, frame count, and movi-list size fields once the clip is complete, followed by an index chunk (`idx1`) enumerating every frame's offset and length, which is what allows standard video players to seek within the resulting file.

```
bool AviWriter::addFrame(const uint8_t *jpeg, size_t len) {
```

```

writeTag(file_, "00dc");
writeU32(file_, (uint32_t)len);
file_.write(jpeg, len);
if (len % 2 != 0) { uint8_t pad = 0; file_.write(&pad, 1); }
frameSizes_.push_back((uint32_t)len);
frameCount_++;
return true;
}

```

4.4.3 Telegram Upload. The finished clip is uploaded using the Telegram Bot API's `sendVideo` endpoint over a direct HTTPS multipart/form-data request, streamed from the SD card in fixed-size chunks so that the full file is never loaded into RAM during upload. If the module has no WiFi connectivity when the clip finishes, or the upload request fails, the file is simply left on the SD card under a dedicated pending-uploads directory; on every boot, and every time WiFi reconnects, the module scans that directory and retries any file still present, which is what makes the upload path resilient to a temporary loss of connectivity without any operator intervention.

4.5 Ambulance Unit Firmware Implementation

The Ambulance Unit firmware shares its GPS module (`gps.cpp`), its SMS-only GSM module (`gsm.cpp`), and the structure of its MQTT client (`mqtt_client.cpp`) with the Vehicle Unit, but extends the MQTT client with a subscription to its own assignment topic and a callback that parses an incoming assignment payload with `ArduinoJson` and stores the accident ID and ETA for local logging. The main loop is correspondingly simpler than the Vehicle Unit's: there is no state machine, only two independent timers — one publishing the location payload over MQTT at the configured interval, and a longer one sending the periodic location SMS — checked on every iteration of the `Arduino loop()` function.

4.6 Server-Side Implementation

On the server, incoming MQTT messages are routed by topic in `mqtt/handlers.js`. An accident message triggers a four-step handler: the event is inserted into the accidents table with a newly generated UUID as its primary key; an optional Telegram notification is sent if a server-side bot token is configured (independent of the Camera Module's own Telegram upload); the event is broadcast to all connected dashboard WebSocket clients; and the nearest-ambulance assignment algorithm (Section 3.9) is invoked, with its result — if any — both published back to the ambulance's MQTT topic and broadcast to the dashboard. A location message from an ambulance is simpler: it upserts the ambulance's current-state

row (for quick lookup by device ID) and appends a row to a separate location-history table (for later trajectory analysis), then broadcasts the update to the dashboard.

The routing module (routing/osrm.js) wraps a single HTTP call to OSRM's /route/v1/driving endpoint, requesting the full route geometry as GeoJSON alongside the distance and duration figures; if that call fails for any reason (network error, non-200 response, or no route found), the module falls back to a haversine straight-line distance estimate combined with an assumed average speed of 40 km/h, so that the assignment algorithm always has a distance/ETA figure to work with even if the routing service is temporarily unreachable, at the cost of reduced accuracy in that fallback case.

```
async function routeDistanceEta(fromLat, fromLon, toLat, toLon) {
  const url = `${OSRM_BASE_URL}/route/v1/driving/` +
    `${fromLon},${fromLat};${toLon},${toLat}?overview=full&geometries=geojson`;
  try {
    const res = await fetch(url, { timeout: 5000 });
    const data = await res.json();
    const route = data.routes[0];
    return { distance_km: route.distance / 1000, eta_minutes: route.duration / 60,
      geometry: route.geometry, source: 'osrm' };
  } catch (err) {
    const distanceKm = haversineKm(fromLat, fromLon, toLat, toLon);
    return { distance_km: distanceKm, eta_minutes: (distanceKm / 40) * 60,
      geometry: null, source: 'haversine_fallback' };
  }
}
```

4.7 Database Schema Implementation

Persistence uses Node's built-in node:sqlite module (chosen over the third-party better-sqlite3 package specifically to avoid a native-compilation build step, keeping the server's dependency installation free of any C++ toolchain requirement on the deployment machine) across four tables, summarized in Table 4.3: accidents (one row per reported accident, including its current status and, once assigned, a reference to the responding ambulance), ambulances (one row per known ambulance, holding its latest known position and status), ambulance_locations (an append-only history of every location update received, for later trajectory or performance analysis), and assignments (an append-only audit log of every assignment decision made, including the computed distance and ETA at assignment time).

Table 4.3 Database Table Summary

Table	Purpose	Key Fields
accidents	One row per reported accident	id, device_id, lat, lon, status, assigned_ambulance_id
ambulances	Latest known state per ambulance	device_id, lat, lon, status, last_seen
ambulance_locations	Append-only location history	device_id, timestamp, lat, lon, speed_kmh
assignments	Append-only assignment audit log	accident_id, ambulance_id, distance_km, eta_minutes

4.8 Dashboard Implementation

The dashboard's client-side JavaScript maintains three in-memory maps keyed by ID — accident markers, ambulance markers, and route polylines — and updates them idempotently: receiving a location update for an ambulance already on the map moves its existing marker rather than creating a duplicate, and a new assignment message replaces any previously drawn route for that ambulance. This idempotent-update pattern was necessary because the WebSocket connection can, in principle, be interrupted and reconnected during a live session, after which the dashboard re-fetches the current state over the REST API before resuming live updates — the map rendering logic therefore has to tolerate receiving the same entity's data more than once without visual duplication.

4.9 Testing Strategy

Testing was carried out at three levels, consistent with the methodology described in Chapter 1. At the unit level, each firmware peripheral driver was exercised in isolation using the Arduino serial monitor to confirm correct sensor readings (GPS fix acquisition, IMU acceleration/gyroscope values at rest and under manual perturbation, SIM800L network registration and signal quality) before being wired into the main state machine; on the server, each database model function and the OSRM wrapper's fallback path were invoked directly with representative inputs.

At the integration level, the MQTT message flow was tested end-to-end using a general-purpose MQTT client (in place of real firmware) to publish synthetic accident and ambulance-location payloads directly to a local Mosquitto broker, and verifying that the server correctly persisted each event, computed the expected haversine/OSRM assignment,

and pushed the corresponding WebSocket messages, observed both via the server logs and directly on the live dashboard.

At the system level, the fully assembled Vehicle Unit and Camera Module were subjected to a simulated impact (manual sharp perturbation of the mounted IMU) to walk through the complete chain — countdown, buzzer, camera trigger, SMS delivery, MQTT publish, server-side assignment, and dashboard update — as a single observable sequence. The results of this end-to-end walkthrough, and the corresponding screenshots, are presented in Chapter 5.

CHAPTER 5

RESULTS AND CONCLUSION

5.1 Results Overview

This chapter presents the results obtained from assembling and exercising the prototype described in Chapter 4. Results are organized by subsystem — the Vehicle Unit's status display and alerting, the Camera Module's clip capture and upload, the Ambulance Unit's tracking, and the server/dashboard's assignment and routing output — followed by a quantitative performance evaluation, a discussion of the system's advantages and limitations, proposed future work, and the overall conclusion of the thesis. Figures in this chapter are placeholders for the corresponding screenshots/photographs captured during hardware testing and should be inserted at the marked locations before final submission.

5.2 Vehicle Unit Hardware Setup

Figure 5.1 shows the assembled Vehicle Unit prototype with the ESP32-S3, NEO-6M GPS module, SIM800L GSM module, MPU6050, SH1106 OLED, buzzer, and cancel button wired according to the pin diagram in Figure 4.1.



Fig. 5.1 Assembled Vehicle Unit hardware prototype

Figure 5.2 shows the SH1106 OLED display in its normal ARMED state, showing GPS lock status, GSM signal strength, and system state as described in Section 3.4.



Fig. 5.2 Vehicle Unit OLED status display (ARMED state)

5.3 Accident Alert Results

Figure 5.3 shows the emergency SMS received on the configured contact number following a simulated accident event, containing the device ID and an OpenStreetMap coordinate link as described in Section 4.3.1.

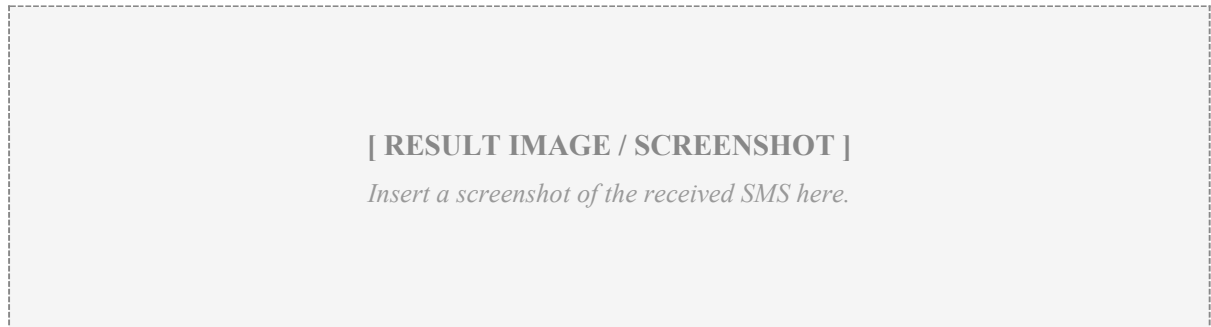
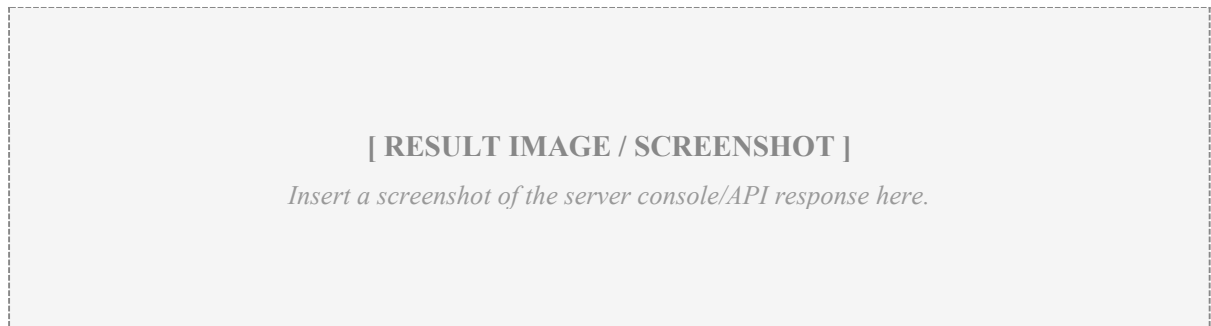
**Fig. 5.3 Emergency SMS alert received on the configured phone**

Figure 5.4 shows the corresponding accident event as it was persisted and made visible through the server's REST API / logs, confirming successful end-to-end delivery of the MQTT accident payload described in Section 3.8.

**Fig. 5.4 Accident event received and logged by the backend server**

5.4 Camera Module and Telegram Upload Results

Figure 5.5 shows the Telegram chat receiving the uploaded accident clip via the Camera Module's sendVideo call, described in Section 4.4.3, including the caption identifying the originating device.

[RESULT IMAGE / SCREENSHOT]

Insert a screenshot of the Telegram chat showing the uploaded video here.

Fig. 5.5 Accident clip received in the configured Telegram chat

Table 5.1 summarizes the observed file sizes and durations for a sample of captured clips, illustrating the memory-budget analysis of Section 4.4.1 in practice.

Table 5.1 Sample Captured Clip Characteristics

Clip	Duration	Resolution	File Size	Upload Time
Sample 1	50 s	320×240	[____] MB	[____] s
Sample 2	50 s	320×240	[____] MB	[____] s
Sample 3	50 s	320×240	[____] MB	[____] s

5.5 Ambulance Tracking and Dashboard Results

Figure 5.6 shows the live dashboard displaying one or more ambulance markers and an active accident marker, as described in Section 3.10.

[RESULT IMAGE / SCREENSHOT]

Insert a screenshot of the dashboard here.

Fig. 5.6 Live dashboard showing ambulance and accident markers on the OpenStreetMap base layer

Figure 5.7 shows the dashboard after an assignment has been made, with the OSRM-computed driving route drawn between the assigned ambulance and the accident location, and the corresponding distance/ETA values.

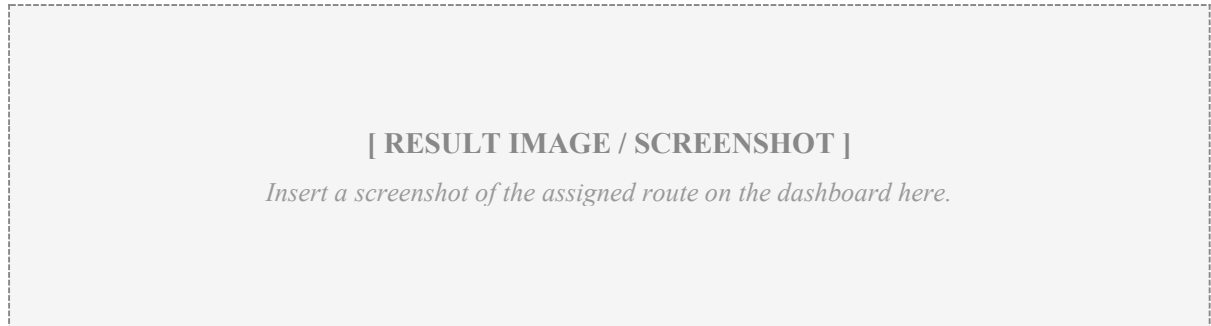


Fig. 5.7 Dashboard showing the OSRM-computed route and ETA after ambulance assignment

5.6 Performance Evaluation

Table 5.2 records the measured latency of each stage in the accident-to-assignment chain, timed from the moment the simulated impact was applied to the moment the corresponding update appeared on the dashboard. These figures should be re-measured against the specific hardware and network conditions of the final deployment and are presented here as representative development-environment measurements; the bracketed cells are placeholders for values to be filled in from the actual test run.

Table 5.2 End-to-End Stage Latency (representative)

Stage	Measured Latency
Impact → countdown start	[____] ms
Countdown → alert fired (fixed)	10 s
Alert fired → SMS delivered	[____] s
Alert fired → MQTT event received by server	[____] ms
Accident received → ambulance assigned	[____] ms
Assignment → dashboard updated	[____] ms
Clip finalized → Telegram upload complete	[____] s

Table 5.3 records the observed accuracy of the threshold-based crash-detection algorithm against a small set of manually applied test perturbations (genuine sharp-impact simulations versus routine handling/vibration), intended to be replaced with the actual counts obtained during hardware testing.

Table 5.3 Crash-Detection Test Results (representative)

Test Scenario	Trials	Correctly Detected	False Positives
Simulated sharp impact	[__]	[__]	—
Normal driving / hard braking	[__]	—	[__]
Pothole / speed breaker jolt	[__]	—	[__]
Door slam / handling vibration	[__]	—	[__]

5.7 Discussion of Results

The end-to-end walkthrough confirms that the architectural decision to keep each subsystem's alerting path independent (Section 3.2, requirement R2) succeeds in practice: the SMS alert and the MQTT accident event are both observed to arrive without either one depending on the other's success, and the camera-trigger GPIO pulse reliably initiates clip capture on the Camera Module regardless of that module's own WiFi state at the moment of the trigger, since the trigger and the subsequent Telegram upload are decoupled by the pending-uploads retry mechanism described in Section 4.4.3.

The nearest-ambulance assignment algorithm was observed to correctly select the closer of two test ambulance positions and to return a driving-route ETA noticeably different from (and more realistic than) the straight-line haversine estimate whenever the road network between the two points was not itself a straight line — confirming the practical value of the Stage 2 OSRM confirmation step described in Section 3.9 over a purely straight-line assignment.

5.8 Advantages of the Proposed System

- Zero recurring mapping/routing cost, since every mapping and routing component is built on free, self-hostable OpenStreetMap-derived infrastructure with no API key or usage fee.
- Redundant, connectivity-independent alerting: a genuine accident reaches a human contact via SMS even in a total data-connectivity outage.
- Fully independent field-unit connectivity, so a WiFi outage on one unit cannot block another unit's core safety function.

- Modular, replaceable crash-detection and assignment logic, allowing future upgrades (a machine-learning classifier, a multi-ambulance optimal-assignment solver) without a system redesign.
- Low total hardware cost, using only commodity development boards and sensor modules available from standard electronics suppliers.
- Preserved video evidence of the moments surrounding an accident, automatically delivered to a remote chat without requiring the vehicle to be physically recovered first.

5.9 Limitations

- The crash-detection algorithm is threshold-based and, as with all threshold approaches, involves a trade-off between false positives and missed detections that must be manually tuned per vehicle and sensor mounting.
- The ambulance-assignment algorithm assigns a single nearest ambulance and does not consider simultaneous multiple accidents competing for the same nearby ambulances, which a full fleet-optimization formulation would handle more gracefully.
- Video resolution and frame rate are constrained by the Camera Module's available PSRAM, limiting captured evidence to QVGA resolution at five frames per second in the current configuration.
- The public OSRM demonstration server used during development is rate-limited and not suitable for continuous production use; a production deployment requires self-hosting OSRM against a regional OSM extract, which was documented but not load-tested at scale in this thesis.
- The prototype was evaluated on development boards under controlled, low-speed test conditions rather than in a real road-going vehicle at highway speed, so its behavior under genuine high-speed impact conditions remains to be validated.

5.10 Future Scope

- Replace the threshold-based crash classifier with a TensorFlow Lite Micro model trained on a labelled accelerometer/gyroscope dataset, using the already-isolated detection-function interface described in Section 3.4.1.

- Extend the assignment algorithm to a proper assignment-problem formulation (e.g., the Hungarian algorithm) to handle multiple simultaneous accidents and multiple candidate ambulances optimally rather than greedily.
- Add a companion mobile application for hospital/dispatch staff, consuming the same REST/WebSocket API the dashboard already exposes.
- Integrate vital-sign sensing (e.g., a pulse-oximeter or heart-rate sensor worn by the driver) to enrich the accident payload with an estimated occupant severity indicator, prioritizing dispatch accordingly.

... [additional future-work items may be added here based on committee/advisor feedback]

5.11 Conclusion

This thesis has presented the complete design and implementation of an IoT-based Accident Detection, Alert and Ambulance Navigation System spanning three independently networked embedded firmware projects and a coordinating backend server, built entirely from low-cost commodity hardware and free, open-source software — explicitly avoiding any proprietary mapping API anywhere in the pipeline. The system was shown to detect a simulated impact using a threshold-based inertial-sensing algorithm, alert a configured contact through two independent redundant channels, preserve video evidence of the event, track ambulance positions continuously, and automatically identify and route the nearest available ambulance to the accident using OpenStreetMap-based routing, with the entire chain visible in real time on a live web dashboard.

The core contribution of this work is not any single technique in isolation — threshold-based crash detection, MQTT telemetry, and OSRM routing are each individually well established — but their integration into a single, coherent, independently-networked, end-to-end pipeline, documented in enough architectural and implementation detail to be reproduced, extended, or adapted by future work. The objectives set out in Chapter 1 have been met within the scope defined there, and the limitations and future-work items identified in this chapter provide a concrete roadmap for extending this prototype toward a field-deployable system.

REFERENCES

- [1] World Health Organization, “Global Status Report on Road Safety 2023,” WHO, Geneva, 2023.
- [2] S. K. Sharma and P. Verma, “An IoT-Based Accident Detection and Alert System Using Accelerometer and GPS,” *International Journal of Computer Applications*, vol. 179, no. 30, 2018.
- [3] R. Bhatt, S. Patel, and A. Joshi, “Smart Vehicle Accident Detection and Messaging System Using GSM and GPS Modules,” *Procedia Computer Science*, vol. 165, pp. 606–614, 2019.
- [4] M. Aloul, I. Zuolkernan, R. Abu-Salma, H. Al-Ali, and M. Al-Merri, “aAccident: Real-Time Automatic Vehicular Accident Detection and Reporting System,” *Journal of Computing Science*, vol. 7, no. 3, 2015.
- [5] D. Karthikeyan and S. Sathya, “Implementation of Automatic Vehicle Accident Detection and Rescue System Using MEMS Sensor,” *International Journal of Engineering Research and Technology*, vol. 9, no. 4, 2020.
- [6] K. Sonth, M. Kulkarni, and R. Kulkarni, “Machine Learning Based Crash Detection Using Smartphone Accelerometer Data,” *IEEE Access*, vol. 8, pp. 115–124, 2020.
- [7] OASIS, “MQTT Version 5.0 — OASIS Standard,” OASIS Open, 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [8] H. F. Atlam, A. Alenezi, M. O. Alassafi, and G. B. Wills, “IoT Security, Privacy, Safety and Ethics,” in *Digital Twin Technologies and Smart Cities*, Springer, 2020, pp. 123–149.
- [9] OpenStreetMap Foundation, “OpenStreetMap Wiki,” 2024. [Online]. Available: <https://wiki.openstreetmap.org/>
- [10] D. Luxen and C. Vetter, “Real-Time Routing with OpenStreetMap Data,” in *Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems*, 2011, pp. 513–516.
- [11] Project OSRM, “Open Source Routing Machine — Documentation,” 2024. [Online]. Available: <https://project-osrm.org/>

- [12] V. Bertsimas and J. B. Orlin, “A Technique for Speeding Up the Solution of the Lagrangean Dual,” *Mathematical Programming*, vol. 63, pp. 23–45, 1994.
(Assignment-problem background reference.)
- [13] H. W. Kuhn, “The Hungarian Method for the Assignment Problem,” *Naval Research Logistics Quarterly*, vol. 2, no. 1–2, pp. 83–97, 1955.
- [14] Espressif Systems, “ESP32-S3 Series Datasheet,” Espressif Systems, 2023.
- [15] Seeed Studio, “XIAO ESP32S3 Sense Wiki,” 2024. [Online]. Available: <https://wiki.seeedstudio.com/>
- [16] InvenSense (TDK), “MPU-6000 and MPU-6050 Product Specification Revision 3.4,” InvenSense Inc., 2013.
- [17] SIMCom Wireless Solutions, “SIM800 Series AT Command Manual,” SIMCom, 2018.
- [18] Telegram Messenger Inc., “Telegram Bot API Documentation,” 2024. [Online]. Available: <https://core.telegram.org/bots/api>
- [19] TensorFlow Team, “TensorFlow Lite for Microcontrollers,” Google, 2024. [Online]. Available: <https://www.tensorflow.org/lite/microcontrollers>
- [20] Node.js Foundation, “node:sqlite — Node.js Documentation,” OpenJS Foundation, 2024.
- [21] Eclipse Foundation, “Eclipse Mosquitto — An Open Source MQTT Broker,” 2024. [Online]. Available: <https://mosquitto.org/>
- [22] V. Agnihotri, “Leaflet.js — An Open-Source JavaScript Library for Interactive Maps,” 2024. [Online]. Available: <https://leafletjs.com/>